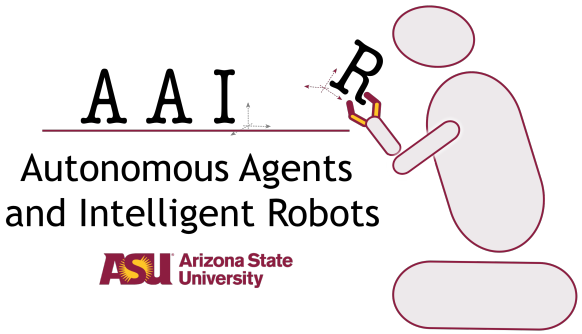
SAT based Planning

Pulkit Verma

Autonomous Agents and Intelligent Robots Lab
Arizona State University

06 February 2019



Outline

Boolean Formula

Any boolean formula consists of

1. Variables that can have value 0 or 1. (For ex, x_1, x_2, x_3)
2. Operators:
   1. And: $x_1 \wedge x_2$.
   2. Or: $x_1 \vee x_2$.
   3. Not: $\overline{x_1}$

Satisfiability

A formula is satisfiable if it is possible to find an interpretation that makes the formula true.

Complexity

Satisfiability is an NP-complete problem

Boolean Satisfiability

Definition

The problem of determining if there exists an interpretation that satisfies a given Boolean formula.

Input

Boolean Formula with n variables $x_1, x_2, \ldots, x_n$.

Problem

Can you assign True/False to the n variables so that the formula becomes True.

Boolean Satisfiability Example

$$(x_1 \vee x_3) \wedge (x_1 \wedge (x_2 \vee \overline{x_3})) \wedge (\overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee \overline{x_2})$$

Boolean Satisfiability Example

$$(x_1 \vee x_3) \wedge (x_1 \wedge (x_2 \vee \overline{x_3})) \wedge (\overline{x_2} \vee \overline{x_3}) \wedge (\overline{x_1} \vee \overline{x_2})$$

$$x_1 = \text{True}$$
$$x_2 = \text{False}$$
$$x_3 = \text{False}$$

Conjunctive Normal Form (CNF)

Using the laws of Boolean algebra, every propositional logic formula can be transformed into an equivalent conjunctive normal form.

Example

$$(x_1 \wedge y_1) \vee (x_2 \wedge y_2) \vee \cdots \vee (x_n \wedge y_n)$$

$$\begin{aligned}
&(x_1 \vee x_2 \vee \cdots \vee x_n) \wedge (y_1 \vee x_2 \vee \cdots \vee x_n) \wedge \\
&(x_1 \vee y_2 \vee \cdots \vee x_n) \wedge (y_1 \vee y_2 \vee \cdots \vee x_n) \wedge \ldots \wedge \\
&(x_1 \vee x_2 \vee \cdots \vee y_n) \wedge (y_1 \vee x_2 \vee \cdots \vee y_n) \wedge \\
&(x_1 \vee y_2 \vee \cdots \vee y_n) \wedge (y_1 \vee y_2 \vee \cdots \vee y_n) \wedge
\end{aligned}$$

Planning as SAT

Idea

The idea is to convert the planning problem into SAT and use off the shelf solvers.

- Planning problem expressed as propositional logic formula.
- Satisfying assignment to the propositional variables will give the plan.
- Propositional variables are called fluents; predicates that change value over time.

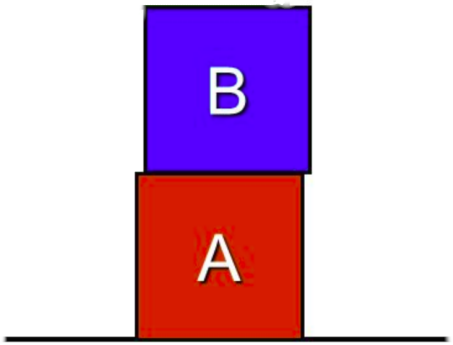
Linear Encoding

- Add a time parameter to fluents.
- Express relations between actions, preconditions, and effects.

Linear encoded SAT Formula

Initial State S_0

- For all fluents f_0 at time t_0, if $f_0 \in S_0$, then f_0 is a clause, else $\neg f_0$ is a clause.



$$On(B, A, t_0) \wedge$$
$$Clear(B, t_0) \wedge$$
$$\neg Clear(A, t_0)$$

Goal State $G = S_k$

- Assume at most k steps.
- Add a fluent f_k for every goal proposition $f \in G$.

Outline

The Flashlight Problem[1]

AIM: Putting 2 batteries into a flashlight.

Predicates

On (cap, flashlight), *In (battery, flashlight)*

Instances

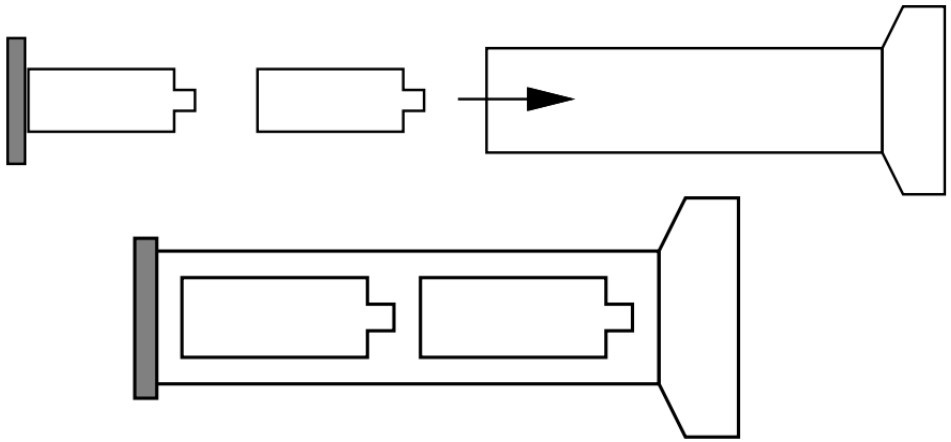
B1, B2, C, F

Actions

Action	Precondition	Effect
PlaceCap	$\neg On(cap, flashlight)$	$On(cap, flashlight)$
RemoveCap	$On(cap, flashlight)$	$\neg On(cap, flashlight)$
Insert (battery)	$\neg On(cap, flashlight)$, $\neg In(battery, flashlight)$	$In(battery, flashlight)$

[1] Planning Algorithms. Steven M. LaValle. 2006.

The Flashlight Problem



Initial State

$\mathsf{I} = On(C, F)$

Goal State

$\mathsf{G} = On(C, F),\ In(B1, F),\ In(B2, F)$

The Flashlight Problem's Solution

Plan

1. *RemoveCap*
2. *Insert*(*B*1)
3. *Insert*(*B*2)
4. *PlaceCap*

How will a SAT solver solve it?

Outline

State Encodings

Initial State

- Do a conjunction of all literals in initial state (I) , including negation of literals not in I.
- Tag all these with time=1.
- **Flashlight Problem**:
  $I = On(C, F, 1) \wedge \neg In(B1, F1, 1) \wedge \neg In(B2, F, 1)$

Goal State

- Conjunction of all literals in Goal (G)
- Tag all these with index F = K+1
- **Flashlight Problem**: Assume we know that solution is achieved at K=4.
  $G = On(C, F, 5) \wedge In(B1, F, 5) \wedge In(B2, F, 5)$

Action Encoding

- Each action is copied over all the time stages.
- For each action $a \in A$, let a_k denote the action applied at time k.
- Do a conjunction over all actions at all time stages.
- Assume preconditions for an action a_k consisting of literals $p_1, p_2, \ldots, p_m$.
- Assume effects for an action a_k consisting of literals $e_1, e_2, \ldots, e_n$.

$$a_k \Rightarrow (p_{1,k} \wedge p_{2,k} \wedge \cdots \wedge p_{m,k} \wedge e_{1,k+1} \wedge e_{2,k+1} \wedge \cdots \wedge e_{n,k+1})$$

Action Encoding Example

We use the fact that $a \Rightarrow b$ is equivalent to $\neg a \vee b$.

Flashlight Problem's Action Encoding (at time k)

$$(\neg PlaceCap(k) \vee (\neg On(C, F, k) \wedge On(C, F, k+1))) \wedge$$

$$(\neg RemoveCap(k) \vee (On(C, F, k) \wedge \neg On(C, F, k+1))) \wedge$$

$$(\neg Insert(B1, k) \vee (\neg On(C, F, k) \wedge \neg In(B1, F, k) \wedge In(B1, F, k+1)) \wedge$$

$$(\neg Insert(B2, k) \vee (\neg On(C, F, k) \wedge \neg In(B1, F, k) \wedge In(B1, F, k+1))$$

We have to encode this for $k = 1, 2, 3, 4$, then use a conjunctions of all those to get the complete action encoding.

Frame Axioms I

We must specify those literals also which does not change from time stage t to time stage $t + 1$.

Full (classical) frame axioms

- Encode the implicit assumption that every literal that is not an effect of an action remains unchanged.
- If a literal l_k was true at time k, and an action a_k that does not affect l_k was executed at time k, the l_{k+1} is true at time $k + 1$.
  $(l_k \wedge a_k) \Rightarrow l_{k+1}$
- Full frame axioms also require that at-least-one action is executed at each time stage. So if there are j possible actions:
  $(a_{k,1} \vee a_{k,2} \vee \cdots \vee a_{k,j})$

Frame Axioms II

Explanatory frame axioms

- For each time step, we encode for every literal that if it is changed, it must be because of some action that changed it.
- Like the previous case, assume that a literal l_k was true at time k, but was changed to $\neg l_{k+1}$ at time $k+1$, and there are j actions that change l_k to l_{k+1}, then one out of them must have been executed at time k.

$$(l_k \wedge l_{k+1}) \Rightarrow (a_{k,1} \vee a_{k,2} \vee \cdots \vee a_{k,j})$$

Conflict Exclusion

The inherent definition of explanatory frame actions allow parallel actions (notice the $\vee$), so we need conflict exclusion to avoid parallel execution of conflicting actions.

Conflict Exclusion

- Two actions are conflicting if the precondition of one of them is negation of effect of the other.
- If two actions $a_{k,1}$ and $a_{k,2}$ are conflicting, then we add the clause $\neg(a_{k,1} \wedge a_{k,2})$ to enforce exclusion.

Why not needed with Full frame axioms?

- Unlike exploratory axioms, they enforce listing all literals (including unchanged ones).
- So 2 actions can happen at same time only if they have same preconditions and effects.
  $(\neg a_{1,k} \vee (p_{1,k} \wedge e_{1,k+1})) \wedge (\neg a_{2,k} \vee (p_{2,k} \wedge e_{2,k+1}))$
- Since $p_{i,k}$ and $e_{i,k+1}$ list all literals, both actions need to have same p and e for this formula to be true.

Complete Exclusion Axiom

- Can be used to add the constraint that only one action is performed at each time stage.
- For any pair of actions $a_{k,1}$ and $a_{k,2}$, we can add the clause $\neg(a_{k,1} \wedge a_{k,2})$ to enforce that only one of them is executed.
- Notice that it is same as conflict exclusion constraint. So we need not apply conflict exclusion if we are applying complete exclusion.

Explanatory Frame Axioms Example

at any given time k:

Flashlight Problem's frame axioms

$$((On(C,F,k) \vee \neg On(C,F,k+1)) \vee PlaceCap(k)) \wedge$$

$$((\neg On(C,F,k) \vee On(C,F,k+1)) \vee RemoveCap(k)) \wedge$$

$$((In(B1,F,k) \vee \neg In(B1,F,k+1)) \vee Insert(B1,k)) \wedge$$

$$(\neg In(B1,F,k) \vee In(B1,F,k+1)) \wedge$$

$$((In(B2,F,k) \vee \neg In(B2,F,k+1)) \vee Insert(B2,k)) \wedge$$

$$(\neg In(B2,F,k) \vee In(B2,F,k+1))$$

We have to encode this for $k = 1, 2, 3, 4$, then use a conjunctions of all those to get the complete action encoding.

Complete Exclusion Axioms Example

at any given time k:

Flashlight Problem's complete exclusion axiom

$$(\neg RemoveCap(k) \vee \neg PlaceCap(k)) \wedge$$
$$(\neg RemoveCap(k) \vee \neg Insert(B1, k)) \wedge$$
$$(\neg RemoveCap(k) \vee \neg Insert(B2, k)) \wedge$$
$$(\neg PlaceCap(k) \vee \neg Insert(B1, k)) \wedge$$
$$(\neg PlaceCap(k) \vee \neg Insert(B2, k)) \wedge$$
$$(\neg Insert(B1, k)) \vee \neg Insert(B2, k))$$

We have to encode this for $k = 1, 2, 3, 4$, then use a conjunctions of all those to get the complete action encoding.

SAT Encoding (sequential)

(Action Encoding) $\wedge$ (Frame Axiom Encoding) $\wedge$
(Complete Exclusion Axiom Encoding)

Parallel Encodings I

Complete Exclusion Axiom encoding not used.

$\forall$ - step plans

- A set of actions can be simultaneous if they are pairwise independent.
- Actions can be executed in any order as long as they are independent.
- Used in the original "Planning as Satisfiability" work.

Parallel Encodings II

$\exists$ - step plans

- A set of actions can be simultaneous if they can be totally ordered so that an earlier action does not disable a later action or change its (conditional) effects.
- Actions $a_{k,1}$ and $a_{k,2}$ can get executed in parallel without the preconditions of both of them being true at time k.
- As long as these actions can satisfy each other's preconditions at some time later.
- Actions can be executed in at least one order.

Outline

SAT Decision procedures

- We consider only bounded planning problems.
- We solve a SAT encoding (formula) using satisfiability decision procedure.

Sound procedure

"A satisfiability procedure is *sound* if every input formula on which it returns a model is satisfiable." [a]

[a] **Automated Planning:Theory & Practice**, M. Ghallab, D. Nau, P. Traverso

Complete procedure

"A satisfiability procedure is *complete* if it returns a model on every satisfiable input formula." [a]

[a] **Automated Planning:Theory & Practice**, M. Ghallab, D. Nau, P. Traverso

Types of SAT Decision Procedures

Systematic Procedures

- Based on *David Putnam procedure*[a].
- Sound and complete.

[a] Actually a refinement of DP procedure, called DPLL (Davis-Putnam-Logemann-Loveland algorithm) is used everywhere

Stochastic Procedures

- Based on idea of randomized local search.
- Sound but not complete.
- Sometimes scale better than complete algorithms to large problems if a solution exists.

David Putnam Procedure - Terminology

Input

Propositional formula Φ in Conjunctive Normal Form

Output

Model $\mu \neq \emptyset$ if Φ is satisfiable, else $\mu = \emptyset$

Unit Clause

A clause with one literal

Unit propagation

For any unit clause in the formula, if literal is positive (negative), unit propagation sets the literal to true (false) and simplifies.

David Putnam Procedure[2]

Davis-Putnam(Φ,μ)
    Unit-Propagate(Φ,μ)
    if $\emptyset \in \Phi$ then return
    if $\Phi = \emptyset$ then exit with μ
    select a variable P such that P or $\neg P$ occurs in ϕ
    Davis-Putnam($\Phi \cup \{P\}$,μ)
    Davis-Putnam($\Phi \cup \{\neg P\}$,μ)
end

Unit-Propagate(Φ,μ)
    while there is a unit clause $\{l\}$ in Φ do
        $\mu \leftarrow \mu \cup \{l\}$
        for every clause $C \in \Phi$
            if $l \in C$ then $\Phi \leftarrow \Phi - \{C\}$
            else if $\neg l \in C$ then $\Phi \leftarrow \Phi - \{C\} \cup \{C - \{\neg l\}\}$
end

[2] Automated Planning:Theory & Practice, M. Ghallab, D. Nau, P. Traverso

DP Example

Problem

$$\Phi = D \wedge (\neg D \vee A \vee \neg B) \wedge (\neg D \vee \neg A \vee \neg B) \wedge (\neg D \vee \neg A \vee B) \wedge (D \vee A)$$

1. Unit-Propagate will remove D and assign it *True*.
   $\mu = \{D\}$, $\Phi = (A \vee \neg B) \wedge (\neg A \vee \neg B) \wedge (\neg A \vee B)$
2. Assume, variable selection chooses A, Davis-Putnam$(\Phi \cup A, \mu)$ calls Unit-Propagate, which eliminates A.
   $\mu = \{D, A\}$
3. Unit-Propagate will return false ($\emptyset \in \Phi$) on next call, so Davis-Putnam will backtrack to choose $\neg A$.
4. Unit-Propagate simplifies Φ to $\neg B$ with $\mu = \{D, \neg A\}$
5. One more step will return the empty set to give final $\mu = \{D, \neg A, \neg B\}$.

Davis-Putnam Procedure

- It is a depth first search through space of all possible assignments.
- If it cannot find an unit literal, it chooses a variable and grounds its value.

Importance of variable selection

- This is a non-deterministic step.
- Non determinism in terms of assigning True or False to P.
- Choosing a variable can be a heuristic in itself.
  - Select the first remaining variable.
  - Select a variable in clause of minimal length.
  - Select variable with maximum occurrences in minimum-size clause.
- The main idea here is to eliminate clauses as early as possible.

Conflict-Driven Clause Learning

- State of the art technique used by recent SAT solvers.
- Similar to DPLL algorithm with one major difference.
- DPLL backtracks chronologically, whereas CDCL backtracks non-chronologically.
- It backtracks to the first assigned variable involved in the conflicting clause.

Stochastic Decision Procedures

- DP procedure used partial assignments at each step.
- Stochastic procedures begin the solution with random total assignments.
- If its not a model, perform some local or random search to get to a solution.
- Can use some form of cost like number of clauses that are not satisfied.
- Use this cost to improve iteratively. Can get stuck in local minima.
- Most popular method is *WalkSAT*.

Some Stochastic Methods

Local-Search-SAT (GSAT), Interative-Repair-GSAT, RandomWalk-GSAT, WalkSAT, Simulated-Annealing.

WalkSAT

- It randomly picks an assignment μ.
- if it is not satisfied, randomly pick any clause $C \in \Phi$ not satisfied by μ.
- Select randomly a variable to be flipped among two possibilities:
  1. *Non-greedy case*: a random variable in C.
  2. *Greedy case*: The variable in C that leads to greatest number of satisfied clauses when flipped.
- Repeat this till μ does not satisfy Φ

Outline

Planning as Satisfiability

1. for $k = 0, 1, \ldots, K$
2. Encode Planning as SAT problem Φ in k steps.
3. if Φ is satisfiable, then find truth values that satisfies Φ, and return the plan.

Problem

Space and Comptational complexity too high.

Solution

Use it with other approach like planning graphs.
Why? "Planning graph did the best job of simplifying a problem, whilst SAT solvers were quickest at extracting a solution."[a]

[a] H. Kautz and B. Selman. Unifying SAT-based and Graph-basedPlanning.

BlackBox[3]

- Featured in 1998 Planning Competition. One of the best.
- Unified Graph-Plan and SAT.

Idea

1. for $k = 0, 1, \ldots, K$
2. Create a planning graph with k levels.
3. Encode planning graph as satisfiability problem
4. If solution found in some time limit, remove some useless actions and return the plan.

BlackBox's SAT Solvers

- WalkSAT.
- Forwardchecking DPLL-based solver *satz*
- *zChaff*: Chaff algorithm's implementation to solve SAT.

[3] BlackBox Home Page

SATPLAN[4]

- Same idea as Blackbox.
- Different SAT Solvers used as compared to BlackBox.

SATPLAN-04

- Used a DPLL based solver "siege".
- Did not have mutexes in graph-plan.

SATPLAN-06

- Enabled mutex propagation
- Only generated clauses for inferred mutexes for fluents, not for actions.

[4] SATPLAN Home Page

Outline

Parallel Plan Search

- The parallel SAT encodings discussed earlier focussed on actions that can be performed simultaneously.
- Parallel plan search focuses on solving many SAT instances simultaneously.

Rintanen et. al. introduced 3 different approaches for parallel plan search.

Algorithm A - Parallel Plan Search

- Start n SAT solvers simultaneously, solving the planning problem for horizon lengths $1, 2, 3, 4, \ldots, n$.
- If a formula is found satisfiable, we have a plan, and terminate.
- If a formula is found unsatisfiable, start a solver for the shortest plan length not solved yet (first $n+1$, then $n+2$, $n+3$, and so on) so that there are always n SAT instances being solved.

Algorithm B - Parallel Plan Search

- Solve horizon lengths 1,2,3,... in parallel using different SAT solvers.
- Assign different CPU times to each SAT solvers proportional to horizon length.
  CPU assigned $= g * (t - 1)$, where $g < 1$ is a constant.
- The rate at which SAT problems are solved form a decreasing geometric sequence.
- Some finite bound is used for the number of SAT solvers run at any given moment (as determined by available memory.)

Algorithm C - Parallel Plan Search

- Solve horizon lengths 1,2,4,8,16,... in parallel using different SAT solvers, up to some finite length (some hundreds or thousands).
- Assign same CPU times to each SAT solver.
- The rate at which SAT problems are solved form a decreasing geometric sequence.

Comparison

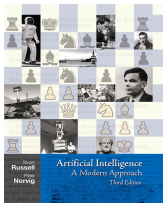
- Algorithms A and B trade the possibility of a constant factor slow-down to a potentially arbitrarily high speed-up.
- Algorithm C does not have any practical performance guarantees w.r.t. A or B or the sequential strategy, but for problems with very long plans it is sometimes much better than A and B.

Madagascar

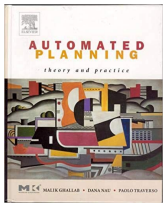
- State of the art SAT solver.
- Used CDCL as systematic decision procedure.
- Implemented algorithm B and C to achieve faster planning
- It uses a bunch of several other heuristics to get improvements.

Outline

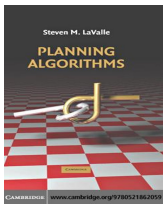
Books



Artificial Intelligence: A Modern Approach. Stuart Russell and Peter Norvig. 2009. Prentice Hall Press, NJ, USA.
Section 10.4.1



Automated Planning: Theory and Practice. Dana Nau, Malik Ghallab, and Paolo Traverso. 2004. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
Chapter 7



Planning Algorithms. Steven M. LaValle. 2006. Cambridge University Press, New York, NY, USA.
Section 2.5.3

Papers

SATPLAN Family

- H. Kautz and B. Selman, Planning as satisfiability, Proceedings of the Tenth European Conference on Artificial Intelligence (ECAI'92), John Wiley, 1992.
- B. Selman, H. Kautz, and B. Cohen. Local Search Strategies for Satisfiability Testing. In Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge, October 11-13, 1993.
- H. Kautz and B. Selman. Unifying SAT-based and Graph-based Planning. 16th International Joint Conference on Artificial Intelligence (IJCAI-99), Stockholm, Sweden, 1999.
- H. Kautz. Deconstructing Planning as Satisfiability. 21st Conference on Artificial Intelligence (AAAI-06), Boston, MA, 2006.
- H. Kautz, B. Selman, and J. Hoffmann. SatPlan: Planning as Satisfiability. Working Notes of the 5th International Planning Competition, Cumbria, UK, 2006.

Papers and Links

Parallel Plans

J. Rintanen, K. Heljanko and I. Niemela, Planning as satisfiability: parallel plans and algorithms for plan search, Artificial Intelligence, 170(12-13), pages 1031-1080, 2006.

Heuristics

J. Rintanen. Planning as satisfiability: heuristics, Artificial Intelligence Journal, 2012.

Reference Page

Planning as Satisfiability - Dr. Rintanen's Page

Thank You